Assignment 3 Report: End-to-End Hugging Face Model Training & Docker Deployment

Name: Kunal Kumar Mishra

Roll No: M25CSA036

Course: DL Ops

Institute: IIT Jodhpur

---

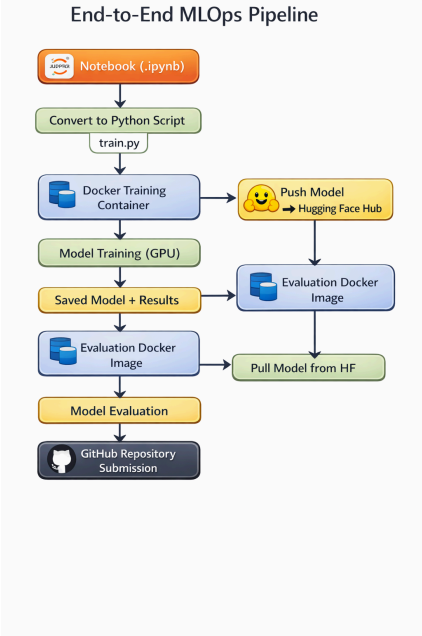
1. Objective

The goal of this assignment was to build and execute a full end-to-end Machine Learning Operations (MLOps) pipeline. This workflow covers model development and validation, containerization with Docker, deployment through the Hugging Face Hub, and experiment tracking with GitHub to maintain reproducibility and ease of deployment across environments.

---

2. Overall Workflow

The complete workflow followed in this assignment is outlined below in the figure:



---

3. Environment Setup Using Docker

Docker was used to create an isolated and reproducible environment independent of the host system.

Steps Performed:

- Created Dockerfile
- Installed dependencies
- Configured working directory
- Enabled GPU support

Docker Build Command:

```
docker build -t hf-train:v1 .
```

Docker Run Command:

```
docker run -it --rm \
--gpus all \
--shm-size=8g \
-v $(pwd):/workspace \
hf-train:v1
```

---

4. Notebook Conversion to Python Script

The instructor-provided notebook was converted into a production-ready Python script.

Steps:

1. Downloaded notebook
2. Converted into Python script
3. Removed notebook artifacts
4. Organized execution flow

Example command used:

```
Bash
jupyter nbconvert --to python notebook.ipynb
```

Final script used: train.py

---

5. Model Selection

A pre-trained transformer model (**DistilBERT**) from Hugging Face was selected.

Reasons for Selection:

- Lightweight architecture
- Faster training compared to standard BERT
- Good performance for text classification tasks
- Efficient fine-tuning capability

Note: The tokenizer and model were loaded directly from Hugging Face.

---

6. Model Training Using Hugging Face Trainer API

The model was fine-tuned using the Hugging Face Trainer API.

Training Configuration:

Parameter	Value
Epochs	3
Batch Size (Training)	10
Batch Size (Evaluation)	16
Learning Rate	2.00E-05
Optimizer	AdamW
Evaluation Strategy	Epoch-wise Evaluation
Device	GPU (CUDA enabled)

Training included:

- Dataset preprocessing
- Tokenization
- Trainer configuration
- GPU-accelerated training

---

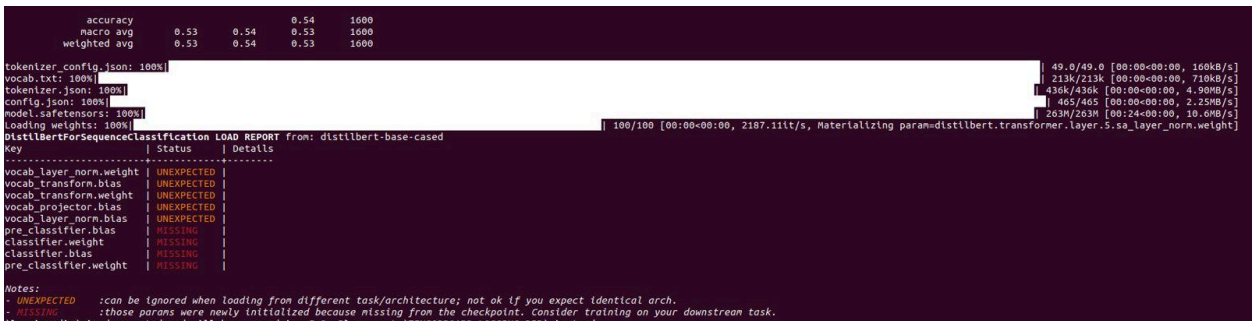
7. Model Evaluation

After training, the model was evaluated on the validation/test dataset.

Metrics Used:

- Accuracy
- F1 Score
- Loss

Example Output:



The results confirmed the successful fine-tuning of the pre-trained model.

---

8. Saving and Uploading Model to Hugging Face

A Hugging Face account was created, and an access token was generated to push the model to the Hub.

Login Command:

python
from huggingface_hub import login
login()

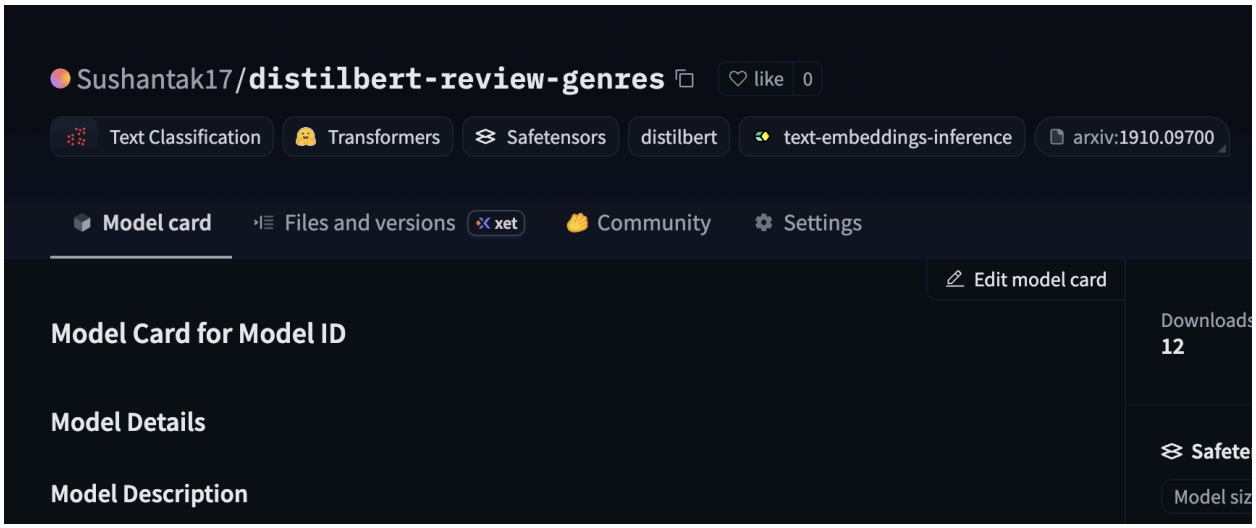
Model Upload:

python
model.push_to_hub("101bytespeedkunal/distilbert-review-genres")

The following artifacts were uploaded:

- Model weights
- Tokenizer
- Configuration files

Hugging Face Model Link: Link



---

9. Re-evaluation from Hugging Face Repository

A separate evaluation Docker container was created, which automatically downloaded the model from Hugging Face and executed the evaluation.

Evaluation Image Build:

Bash
docker build -t hf-eval:v1 -f Dockerfile.eval .

Run Evaluation:

Bash
docker run -it --rm --gpus all hf-eval:v1

Output:

Plaintext
Model loaded successfully from Hugging Face

Observation: The evaluation results were consistent with local training results, confirming correct deployment.

---

10. Final Evaluation Docker Image

A lightweight production Docker image was created specifically for inference purposes.

Purpose:

- Separate training and inference environments
- Establish a reproducible evaluation setup
- Ensure production-ready deployment

---

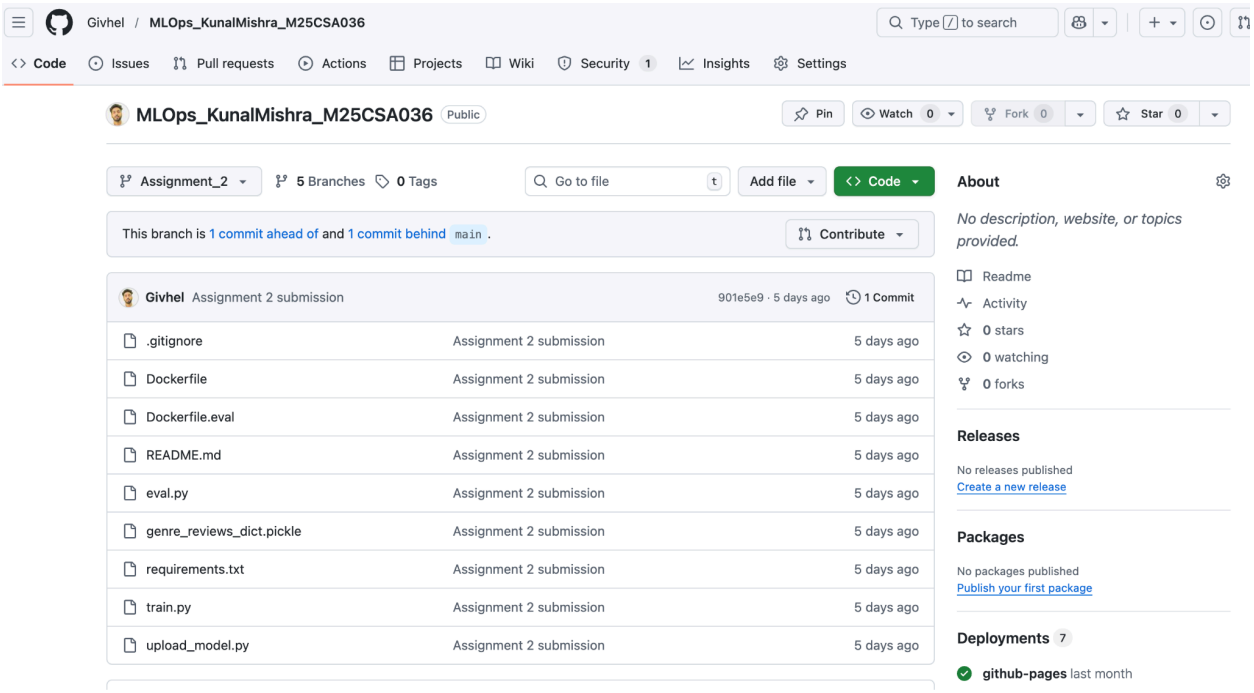
11. GitHub Repository

All project files were version-controlled and pushed to GitHub.

Repository Includes:

- train.py
- eval.py
- Dockerfile
- Dockerfile.eval
- requirements.txt
- README.md

✅ **GitHub Repository Link:** Link



---

12. Challenges Faced

During implementation, several challenges were encountered:

- Dependency conflicts inside Docker containers
- GPU configuration issues

- Missing libraries during container execution
- Docker image size management

These issues were resolved through iterative debugging and environment configuration.

---

13. Key Learnings

This assignment provided practical exposure to:

- End-to-end MLOps workflows
- Docker containerization
- Hugging Face model deployment
- Reproducible ML experiments
- Version control using GitHub
- Separation of training and inference environments

---

14. Conclusion

The assignment effectively showcased the full machine learning lifecycle, from initial experimentation to final deployment. Docker was used to maintain reproducibility, Hugging Face facilitated seamless model sharing, and GitHub ensured organized version control, together establishing a robust and production-ready MLOps pipeline.